33. Deadlocks, Races, and Cancellation Patterns

33. Deadlocks, Races, and Cancellation Patterns

Phase: Phase 5 — Concurrency
Estimated time: 1–2 days
Slug: `33-deadlocks-races`

Learning objectives

- Recognize classic deadlock patterns
- Eliminate data races systematically
- Cancel in-flight work with context
- Write concurrency regression tests

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Deadlocks and races are production killers. Cancellation patterns and the race detector teach you to design for failure.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- Deadlock: all goroutines wait forever — often lock order or channel cycles.
- Race: unsynchronized memory access — undefined behavior.
- Always acquire locks in a consistent global order.
- Propagate context cancellation to break waits.
- Prefer ownership transfer via channels to shared mutable state when possible.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-deadlocks, && cd /tmp/gw-deadlocks,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "context"
    "fmt"
    "time"
)

func main() {
    ctx, cancel := context.WithTimeout(context.Background(), 50*time.Millisecond)
    defer cancel()
    ch := make(chan int)
    select {
    case <-ch:
        fmt.Println("got")
    case <-ctx.Done():
        fmt.Println("cancelled")
    }
}
```

Run it:

```
go run .
```

Expected output:

```
cancelled
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Create a channel deadlock; fix with buffer or goroutine.
2. Run a racy counter under -race.
3. Add timeout select to a blocking receive.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Deadlocks

Common causes: - Lock ordering A then B vs B then A - Two goroutines waiting on each other unbuffered - WaitGroup Add inside the goroutine incorrectly

Races

A data race: concurrent access to a variable, at least one write, no sync. Fix by synchronizing or not sharing.

Cancellation pattern

```
func worker(ctx context.Context, in <-chan Task) error {
    for {
        select {
        case <-ctx.Done():
            return ctx.Err()
        case t, ok := <-in:
            if !ok { return nil }
            if err := handle(ctx, t); err != nil { return err }
        }
    }
}
```

Failure modes to watch while coding

- “It didn’t race on my machine” without -race
- Fixing races by sleeping
- Nested locks without ordering rules

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Deadlocks exist in threading too; Go makes them more visible.
- Race detector is a superpower — use it in CI.
- asyncio cancellation ≈ context cancel.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

On deadlock, print goroutine stacks (SIGQUIT / panic) and look for who waits on what.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Deadlocks, Races, and Cancellation Patterns” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- “It didn’t race on my machine” without -race
- Fixing races by sleeping
- Nested locks without ordering rules

Exercises

Work in `exercises/33-deadlocks-races/`. Read that folder’s `README.md` before you code.

What you will build

Ensure cancellation paths return instead of hanging; fix racey access.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/33-deadlocks-races/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Define data race
2. Give a deadlock example
3. How context prevents abandoned work?

Further reading

- https://go.dev/doc/articles/race_detector